MERCHANT SERVICES · ENTERPRISE ARCHITECTURE

The Payment Operating System

Volume 1 — Enterprise architecture. The component architecture for an intelligent global PSP, with product boundaries drawn over it: what each product owns, what it consumes, and the rules that stop the boundaries eroding under delivery pressure.

Document	EA-001 · v1.1 · Volume 1
Date	26 September 2026
Audience	Product, Architecture, Platform Engineering
Status	For review — 19 decisions open
Components	71 across six layers
Added	17 beyond the original brief

Contents

01 How to read this document

02 Ten principles that hold the architecture together

03 The stack

04 Layer 0 and 1 — client edge and access

05 Layer 2 — the four product domains

06 Layer 3 — shared platform services

07 Layer 4 and 5 — integration fabric and foundation

08 The external ecosystem and the trust boundary

09 The product boundary overlay

10 Payment Processing

11 Credential Vault

12 Payment Instrument Intelligence

13 Authentication Manager and Fraud

14 The eleven named shared services

15 The seventeen additions, and why each one is load-bearing

16 Master component register

17 Twelve boundary rules

18 The nine contentious boundaries

19 Runtime — one authorization, edge to issuer

20 Control plane and data plane

21 Six anti-patterns this design exists to prevent

22 Open decisions — nine architectural, ten strategic

01

How to read this document

> This document answers three questions that are usually conflated: what components exist, who owns each one, and what stops an owner from quietly absorbing someone else's. The third question is the one that decides whether this architecture survives contact with a roadmap.

It is written in three passes over the same material. The first pass is a **component architecture** — six layers, sixty-two components, no ownership claimed. The second pass overlays **product boundaries** on exactly that layout, so the two views are guaranteed to agree. The third pass states the **rules and the contested seams**, because a boundary that is drawn but not defended is a boundary that will be gone in two quarters.

Where a component was not in the original brief it is marked with a `+` in the diagrams and called out in section 15. Seventeen were added. None of them is speculative; each one is either already being done somewhere without an owner, or is a gap that shows up the first time a real client is boarded.

ON THE WORD "PRODUCT"

A *product boundary* here is an ownership and contract boundary, not a deployment boundary and not necessarily a sellable SKU. Payment Processing is one product boundary containing perhaps a dozen services. What makes it one product is that it has one owner, one roadmap, one contract surface and one place where a decision about it gets made.

What is sellable is defined in the Product and Solution Catalog, which is a separate thing entirely and deliberately so. Section 18 explains why conflating the two is the most expensive mistake on this page.

02

Ten principles that hold the architecture together

Every architecture diagram is aspirational until someone writes down what it forbids. These are the constraints that the rest of the document derives from. They are stated as absolutes because a principle with an exception clause is a preference.

#	PRINCIPLE	WHAT IT RULES OUT
1	**One system of record per fact.** Every fact has exactly one owning component. Everyone else holds a cache with a stated staleness tolerance.	Three services that each “know” the merchant’s pricing, disagreeing at month end.
2	**No product reads another product’s store.** Every cross-boundary read is a versioned contract with an owner and a deprecation policy.	The shared database that makes two teams one team without anyone deciding that.
3	**Control plane and data plane are separated.** Anything authored, reviewed and versioned lives in the control plane. The data plane resolves locally and never calls it synchronously.	A configuration service outage that stops authorization.
4	**Policy constrains; strategy chooses.** Policy produces the permitted option set. A strategy selects within it and can never widen it.	Regulatory rules encoded as routing preferences, discovered during an audit.
5	**Products own strategies; the platform owns decisions.** A product author’s strategy is deployed to the Decision Platform; it does not ship its own decision engine.	Four decision engines, four definitions of a control group, no shared evidence.
6	**Capability, product and entitlement are three registries.** What the platform can do, what we sell, and what this client bought are different questions with different lifecycles.	A capability that cannot ship until someone prices it.
7	**Money moves in one place.** No product domain initiates a payout, a fee or a ledger entry directly.	Reconciliation breaks whose origin nobody can trace.
8	**Every decision emits evidence.** What was chosen, what else was permitted, the propensity, and what the fallback would have done — on the decision, not reconstructed after.	A model programme that starts from zero data in eighteen months’ time.
9	**Bitemporal by default for anything contested.** Pricing, policy, entitlement and capability record both what was true and when we knew it.	An inability to answer “what rate was in force on that transaction” during a dispute.
10	**Tenancy and residency are edge concerns.** Routing a request to the right region and legal entity happens once, at the edge, not in each service.	Seventy-one components each with their own interpretation of data residency.

THE TEST FOR A NEW COMPONENT

Before adding anything to this architecture, it must answer: which facts does it uniquely own, which contract does it expose, which existing component would otherwise own this, and what breaks if it is unavailable. A component that cannot name a fact it uniquely owns is a library, not a service.

03

The stack

> Six layers. The two that matter architecturally are L2, where products live, and L3, where everything a product would otherwise build for itself lives instead.



The shape is deliberate. **L2 is thin and L3 is thick.** A product domain in this architecture is a set of strategies, a resource model and a contract surface — not a full stack. If L2 were thick, each product would carry its own decisioning, its own experiment framework, its own policy interpretation and its own metering, and the four product boundaries would drift apart until cross-product work became a negotiation rather than a call.

The cost of this shape is real and worth naming: **L3 becomes a critical dependency for every product roadmap.** A shared service that cannot ship at the pace its consumers need becomes the reason four roadmaps slip, and the political response is always the same — a product team quietly builds its own. Section 17 is largely about preventing that.

LAYER	WHAT LIVES HERE	CHANGES AT THE PACE OF
L0 · Client edge	Everything a client or their developer touches directly	Client demand and channel launches
L1 · Edge and access	Admission, identity, entitlement, idempotency, delivery	Platform-wide contract changes
L2 · Product domains	The four products: their resource models, strategies and contracts	Product roadmaps
L3 · Shared services	Everything more than one product needs	Slowest — these are contracts
L4 · Integration fabric	Connectivity to processors, networks, banks and partners	Scheme mandates and partner onboarding
L5 · Foundation	Runtime, observability, change, residency, continuity	Infrastructure cycles

Layer 0 and 1 — client edge and access

> The edge is where most architectures leak. Every component here is one that a product team will try to own a private copy of, because each of them sits between that team and its client.

Layer 0 — client edge

COMPONENT	RESPONSIBILITY	OWNS
Merchant systems	The client's own ecommerce platform, order management or ERP. Outside our boundary, named because integration shape is driven by it.	Nothing of ours
Checkout and Payment Element	Hosted fields, drop-in UI, payment element. The surface that collects a credential without putting the client in PCI scope.	Rendered session state
Mobile and in-app SDK	Native capture, wallet integration, device signals at source.	SDK version and capability matrix
Terminal and POS estate	The physical fleet: attended, unattended, semi-integrated and softPOS.	Terminal transaction session
Client portal and reporting	Where a merchant sees transactions, settlements, disputes and statements.	Presentation only — no facts
Developer portal and sandbox	Documentation, keys, API explorer, test data and certification.	Published API surface

Layer 1 — edge and access

COMPONENT	RESPONSIBILITY	WHY IT IS HERE AND NOT IN A PRODUCT
API gateway	Admission, versioning, rate limiting, request shaping.	A per-product gateway means a per-product API contract and four deprecation policies.
Client identity and access	Client authentication, keys, OAuth, role and scope resolution.	One credential works across products, or clients integrate four times.
Entitlement enforcement	The decision point that asks whether this client is entitled to this call.	If products enforce entitlement themselves, entitlement diverges from what was sold.
Idempotency and request ledger	Request deduplication and the authoritative record that a request was received.	Sits with Payment Processing because the retry semantics are payment semantics.
Webhook and event delivery	Outbound delivery, ordering, retry, replay and signing.	Four webhook implementations means four retry policies and four client integrations.
Tenancy and residency router +	Resolves legal entity, region and data residency once, at admission.	Added. Without it, sixty-two components each interpret residency independently.

WHY IDEMPOTENCY SITS WITH PAYMENT PROCESSING

It looks like a platform concern and it is usually built as one. It is placed inside the Payment Processing boundary here because the hard part is not deduplication — it is deciding what a repeated request *means* for an in-flight authorization, a partial capture or a retry that already reached the network. That judgement is payment domain knowledge, and a generic platform service will get it wrong in exactly the cases that cost money.

05

Layer 2 — the four product domains

> Four boundaries, each with one owner and one contract surface. The line between them is drawn on what each uniquely knows, not on what each does.

Payment Processing

PP

- Order · Payment · Session · Attempt
- Orchestration (UPOP)
- Routing & Retry
- Capture · Refund · Void
- Network & Processor Connectivity
- APM & Local Rails

Credential Vault

- Surrogate Service (vaultless)
- Credential State Service
- Network Tokenisation (TSP)
- Account Updater
- Consent & Mandate
- Identity Resolution (PAR)

Payment Instrument Intelligence

PII

- BIN & Range Intelligence
- Issuer Intelligence
- Instrument Classification
- Reachability & Eligibility
- Regulatory Attributes
- Cost & Interchange Reference

Authentication Manager & Fraud

- 3DS / SCA Orchestration
- Exemption Engine
- Device & Behavioural Signals
- Fraud Scoring & Models
- Risk Rules
- Fraud Case Management

The cleanest test for which domain owns a fact is the granularity at which it varies:

IF THE FACT VARIES BY…	IT BELONGS TO	EXAMPLE
The transaction or order	Payment Processing	Attempt state, capture amount, retry history
The individual cardholder credential	Credential Vault	Token status, PAR, mandate, account updater result
The BIN or instrument class	Payment Instrument Intelligence	Issuer, product type, regulated debit flag, reachability
The risk of this specific attempt	Authentication Manager and Fraud	Score, device reputation, exemption eligibility

That test resolves most arguments. The two it does not resolve — whether interchange reference data belongs to Instrument Intelligence or to Monetisation, and whether the exemption decision belongs to Authentication or to Payment Processing — are covered in section 18.

Layer 3 — shared platform services

> Twenty-four services in four groups. Eleven were named in the brief; thirteen were added. This layer is the actual product of the platform organisation — the four product domains are its customers.

CONTROL & GOVERNANCE

- **Decision Platform** as a Service
- **Experimentation** Platform as a Service
- **Global Policy** Platform
- **Control Plane &** Capability Catalog
- **Product & Solution** Catalog
- **Scheme & Network** Management +

MONEY

- **Settlement** Platform
- **Sub-ledger &** Reconciliation
- **Money Movement** & Payout +
- **Currency** Management (FX)
- **Metering &** Monetisation
- **Tax Services** +

CLIENT LIFECYCLE

- **Boarding & Servicing** Orchestrator
- **Merchant Risk** & Underwriting +
- **Compliance,** Screening & RegRep +
- **Dispute & Chargeback** Management +
- **Case & Workflow** Management +
- **Document & Contract** Management +

PLATFORM SERVICES

- **Data Platform** as a Service
- **Reference & Master** Data Management +
- **Cryptographic** Services & KMS +
- **Notification &** Communication +
- **Terminal & Device** Management +
- **Entitlement** Service +

\+ added beyond the original brief

The grouping is by ***change cadence and consumer***, not by technology. Control and governance services change when the operating model changes. Money services change when pricing, regulation or a scheme mandate changes. Client lifecycle services change when the commercial motion changes. Platform services change on an infrastructure cycle. Putting a service in the wrong group is usually a sign it is doing two jobs.

THE FAILURE MODE THIS LAYER HAS

A shared service that is slower than its consumers need will be forked, and the fork will be justified as a “thin wrapper” or a “temporary adapter”. Two years later it is a second implementation with its own semantics, and the cost of that shows up as an integration project nobody budgeted.

The only durable defence is that each shared service publishes a capability roadmap its consumers can plan against, and that forking one requires an explicit, named architectural exception rather than a sprint decision. Rule 12 in section 17 states this.

Layer 4 and 5 — integration fabric and foundation

Layer 4 — integration fabric

COMPONENT	RESPONSIBILITY	BOUNDARY
Connector and adapter fabric	One normalised model in, processor-specific wire format out. Certification, versioning and capability declaration per connector.	Payment Processing
Network interface	ISO 8583 and ISO 20022 endpoints, scheme sessions, stand-in behaviour.	Payment Processing
Event backbone	Ordered, durable, replayable internal events. The substrate for the data platform and every asynchronous contract.	Shared
File and batch transfer	Clearing files, settlement files, scheme reports, bulk client files.	Shared
Partner gateway	ISV, marketplace, referral partner and platform integrations.	Shared
Sandbox and simulator fabric +	Deterministic processor, network and issuer simulators, plus reproducible test data.	Shared

Layer 5 — foundation

COMPONENT	RESPONSIBILITY
Observability and SRE	Telemetry, tracing, SLOs, incident tooling. Traces must carry the decision correlation identifier.
Platform runtime and service mesh	Compute, service identity, mTLS, traffic management, multi-region topology.
Secrets and configuration	Secret distribution and rotation. Distinct from the control plane, which holds business configuration.
CI/CD and change management	Build, deploy, approval and audit. The path a code change takes — not the path a strategy takes.
Data residency and sovereignty +	Enforcement of where data may be stored and processed, and proof of it.
Resilience and continuity +	Regional failover, degradation modes, stand-in processing, recovery objectives.

THE DISTINCTION BETWEEN CI/CD AND THE CONTROL PLANE

Both publish change. They are different systems because they have different risk profiles and different approvers. CI/CD moves *code* and is governed by engineering change management. The control plane moves *configuration* — strategies, policies, rates, entitlements — and is governed by whoever owns that business decision.

The entire premise of the decision platform is that a routing change stops being a CI/CD event and becomes a control plane event. If those two paths merge, that premise is gone.

The external ecosystem and the trust boundary

> Six classes of external counterparty. Each one imposes something on the architecture that is not negotiable, and the architecture is partly a record of those impositions.

COUNTERPARTY	WHAT THEY IMPOSE	WHERE IT LANDS
Card networks	Mandates on a fixed calendar, message format, certification, token requirements, routing rules.	Scheme and Network Management; connector fabric
Issuers	Authorization behaviour we cannot observe directly and cannot control. The reason experiments exist rather than assertions.	Instrument Intelligence; Decision Platform evidence
Acquirers and processors	Capability variation, availability, settlement timing, fee structure.	Connector fabric; capability registry; settlement
Banks and clearing	Funding windows, cut-offs, rail-specific finality and reversal semantics.	Money Movement and Payout; settlement
Token service providers	Provisioning, lifecycle events, cryptogram mechanics, PAR issuance.	Credential Vault
Vendors	KYC, sanctions, FX rates, tax determination, dispute networks.	Compliance and Screening; Currency Management; Tax; Dispute Management

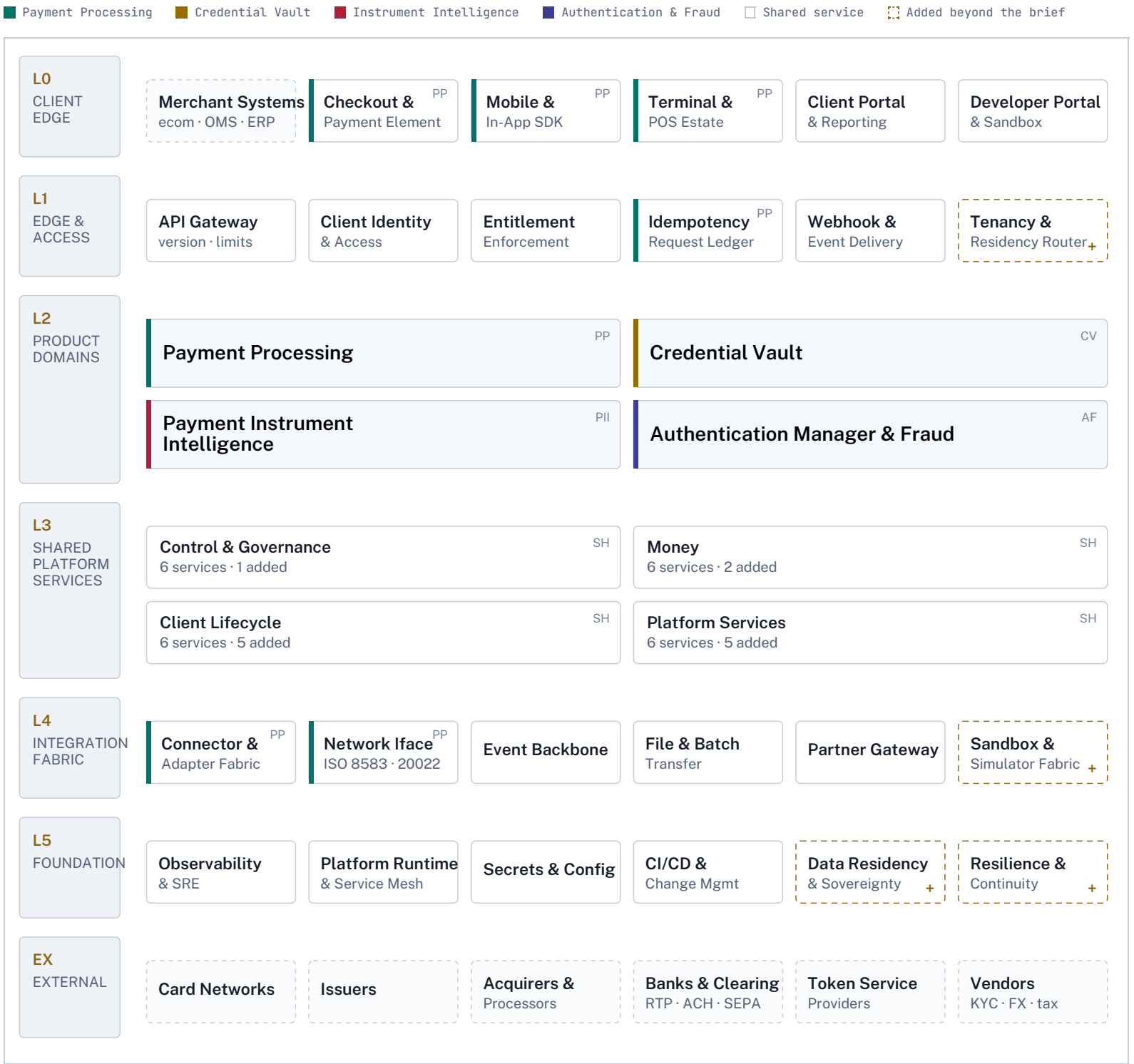
THE TRUST BOUNDARY IS NOT THE NETWORK BOUNDARY

Cardholder data crosses into this architecture at three places and only three: the payment element, the terminal estate and the connector fabric. Everything else operates on surrogates. That is what makes a coherent PCI scope possible, and it is why the Cryptographic Services boundary in section 15 is drawn where it is.

The corollary is a hard rule: **no component outside those three may accept a PAN**, including in a log line, an error payload, a support tool or a test fixture. This is the kind of rule that is true on day one and quietly false by month eighteen unless something enforces it.

The product boundary overlay

> The same layout as section 3, coloured by owner. The point of putting them on the same layout is that the two views cannot drift apart.



Three things fall out of the overlay that are not obvious from a component list.

Payment Processing owns the edge and the wire. It is the only product domain with components in L0, L1 and L4. That is correct — the checkout element, the terminal session, the idempotency semantics and the connector fabric are all payment domain knowledge — but it means Payment Processing carries platform-shaped obligations that the other three do not. It will be the bottleneck for anything that touches the request path, and it should be staffed on that assumption.

The other three products are entirely L2. Credential Vault, Instrument Intelligence and Authentication and Fraud have no components at the edge, in the fabric or in the foundation. They are invoked; they do not sit in the path by themselves. This is what makes them independently replaceable and independently testable, and it is worth defending when one of them asks for its own edge surface.

Everything else is shared, and that is most of the platform. Forty-one of seventy-one components carry no product boundary. If the organisation is shaped around the four product names, two-thirds of the architecture has no natural home — which is the single most common way a platform of this shape fails.

THE ORGANISATIONAL READING OF THIS DIAGRAM

A product-shaped org chart over this architecture leaves L3 orphaned or, worse, distributed across four product teams who each own a slice of it. Both outcomes end the same way: the shared services fragment and the boundaries collapse inward.

The architecture implies a platform organisation that owns L1, L3, L4 and L5 as a product in its own right, with the four domains as its named customers and a published contract with each. That is an organisational commitment, not a diagram.

Payment Processing

> The transaction lifecycle, the orchestration that drives it, and the connectivity that reaches the outside world. The largest boundary, and the only one that spans layers.

What it owns

COMPONENT	SYSTEM OF RECORD FOR	LAYER
Order, Payment, Session, Attempt	The canonical transaction resource model and every state transition within it	L2
Orchestration (UPOP)	Flow execution, the sequence of decision points, and the transaction context	L2
Routing and retry	Processor selection and retry placement — as strategies, executed by the Decision Platform	L2
Capture, refund, void	Post-authorization money operations against an attempt	L2
Network and processor connectivity	Session state, stand-in behaviour and per-connector capability declaration	L2 / L4
APM and local rails	Non-card method orchestration and rail-specific semantics	L2
Checkout and payment element	The collection surface and its session	L0
Terminal and POS estate	The card-present transaction session	L0
Idempotency and request ledger	The authoritative record that a request was received and what it resolved to	L1
Connector and adapter fabric	Wire format translation, certification state, connector versioning	L4
Network interface	Scheme message construction and the ISO endpoints	L4

What it consumes, and must not reimplement

FROM	WHAT	THE RULE
Credential Vault	A presentable credential for this attempt	Payment Processing never stores a PAN or a token
Instrument Intelligence	BIN, issuer, eligibility and cost attributes	It never maintains its own BIN table
Authentication and Fraud	An authentication outcome and a risk decision	It never scores risk itself
Decision Platform	Execution of the strategy bound at each decision point	It never ships a rules engine
Global Policy Platform	The permitted option set for this context	It never encodes regulation as routing logic
Settlement and Sub-ledger	Nothing at authorization time — it emits, it does not call	It never writes a ledger entry

The resource hierarchy

Order, Payment Intent, Attempt and Execution Rail are four levels, not four names for the same thing. The level that earns its keep is the **Payment Intent** — it is what makes split tender, deposits, instalments, marketplace splits and retries expressible without a flag on the order.

THE TRANSACTION RESOURCE HIERARCHY

ORDER — what the buyer is paying for
one per commercial event

PAYMENT INTENT — an intent to collect a specific amount
one or many per order — split tender, deposits, instalments

PAYMENT METHOD — the form the buyer chose
card, wallet, account-to-account, bank

ATTEMPT — one try at moving the money
one or many per intent — retries, cascades, recovery

EXECUTION RAIL — how it actually moves
card network, ACH, RTP, SEPA, stablecoin

WHY THE INTENT LAYER EARNS ITS PLACE

Split tender — two intents, one order	Deposit then balance — two intents, different dates
Instalments — one intent per schedule entry	Marketplace — one intent per seller
Retry and cascade — many attempts, one intent	Partial capture — one intent, several captures
Subscription cycle — one intent per period	Agent purchase — intent carries the mandate

— Without the intent layer, every one of these is modelled as a special case on the order or on the attempt, and the API grows a flag for each.

Payment method as an abstraction, not a branch

A top-tier platform cannot be a card gateway with alternative methods bolted on. The client chooses a *payment method*; the platform chooses an *execution rail*. Keeping those separate is what makes adding a rail a quarter of work rather than a year of it.

PAYMENT METHOD SITS ABOVE THE CARD MODEL, NOT BESIDE IT

PAYMENT METHOD — what the buyer chose · the only thing the API surface exposes

CARD credential, network token, wallet token, MPAN	**WALLET** device token and cryptogram, pass-through	**ACCOUNT TO ACCOUNT** bank authorisation, mandate, push or pull
BANK TRANSFER ACH, SEPA, RTP, FedNow, wire	**DEFERRED** BNPL, instalment provider, invoice terms	**STORED VALUE** balance, gift, loyalty, stablecoin wallet

EXECUTION RAIL — how the money actually moves · never exposed as the client's choice

— A new payment method is a new family member, not a new transaction model. This is the difference between adding a rail in a quarter and adding one in a year.

THE RAIL IS NEVER THE CLIENT'S CHOICE

A client picks "bank transfer"; the platform picks ACH or RTP or FedNow based on amount, urgency, cost, reachability and the strategy bound at that decision point. If the client picks the rail, every rail optimisation we could have made becomes their integration decision and we can never improve it on their behalf.

The exception is where regulation or the client's own commercial agreement requires a specific rail. That is a policy constraint, expressed in the Policy Platform, not a branch in the API.

Internal bounded contexts

This boundary is large enough to become the thing it was meant to prevent. The product boundary is about who decides; the contexts below are about what deploys, scales and fails independently.

ONE PRODUCT OWNER DOES NOT MEAN ONE TECHNICAL SYSTEM

- **Transaction** — order, intent, attempt state
- **Orchestration** — flow execution and context
- **Routing** — processor selection strategies
- **Retry & Recovery** — retry placement and cascade
- **Capture & Refund** — post-authorization operations
- **Payment Methods** — method family semantics
- **Connector Runtime** — wire translation and certification
- **Network Interface** — scheme sessions and ISO
- **Terminal Session** — card-present capture
- **Idempotency** — request identity and resolution

— Ten internal bounded contexts under one product boundary. The boundary is who decides; the contexts are what deploys. Collapsing the two is how a clean product boundary becomes the new monolith.

THE MOST LIKELY EROSION

Payment Processing sits on the request path, so every latency problem in the architecture presents first as a Payment Processing problem. The reflex fix is to cache another product's data locally, then to keep it warm, then to decide when it is stale, then to own it.

That path has been walked by every orchestration platform that ended up containing a BIN table. The defence is a stated staleness tolerance per cached fact, with the owning product responsible for distribution — not a local copy Payment Processing maintains.

Credential Vault

> Everything that is true of a specific cardholder credential. Split internally into a stateless surrogate service and a stateful credential service, because those two have opposite scaling and availability properties.

COMPONENT	SYSTEM OF RECORD FOR	NOTE
Surrogate service	Nothing — it is a pure function	Vaultless. Derivation, key epoch, no storage. Sub-millisecond, no replication lag.
Credential state service	Token status, expiry, provisioning state, channel provenance	Stateful and unavoidably so. Network tokens are assigned, not derived.
Network tokenisation	The TSP relationship and token lifecycle events	TRID, cryptogram provisioning, suspend and resume
Account updater	The latest known good credential and the update history	Both scheme updater and issuer-direct
Consent and mandate	What the cardholder agreed to, when, and in which channel	Underpins MIT, subscriptions and SCA exemption claims
Identity resolution	The mapping from a captured credential to a stable identity	PAR first, deterministic surrogate second

THE SPLIT IS THE POINT

Vaultless applies to the surrogate and to nothing else. A network token identifier is issued by the network and cannot be derived from the PAN; neither can token status, PAR, expiry or provisioning state. Any proposal that reads “go vaultless and retire the vault” has missed this.

What vaultless actually buys is that the ***surrogate*** leaves the hot path as a stateful dependency — which is the majority of vault calls by volume, and all of the card-present ones where the latency budget is tightest.

Cross-channel identity, and the one trap

The resolution ladder is PAR, then deterministic surrogate, then merchant-scoped linkage. The trap is that a wallet DPAN and its funding FPAN are different PANs and therefore derive different surrogates. Only PAR links them. An omnichannel identity built on the surrogate alone will treat every wallet tap in store as a new customer.

Payment Instrument Intelligence

> Everything that is true of an instrument ***class*** rather than an individual credential. The reference layer the rest of the platform reasons over.

COMPONENT	SYSTEM OF RECORD FOR
BIN and range intelligence	BIN ranges, product codes, issuer mapping, range changes over time
Issuer intelligence	Observed issuer behaviour: approval patterns, authentication posture, stand-in behaviour
Instrument classification	Credit, debit, prepaid, commercial, consumer, regulated status
Reachability and eligibility	Which networks, rails and token services this instrument can actually be presented on
Regulatory attributes	Regulated debit status, cross-border flags, issuer country, scheme-specific classifications
Cost and interchange reference	Interchange qualification reference data and scheme fee structures

THE LINE AGAINST CREDENTIAL VAULT

If the fact varies per cardholder, it is Vault. If it varies per BIN range or instrument class, it is Instrument Intelligence. A specific token's status is Vault. Whether that BIN supports network tokens at all is Instrument Intelligence.

The boundary is tested by the account updater: the ***result*** of an update is a per-credential fact and belongs to the Vault, but the observation that a BIN range has been reissued is a class-level fact and belongs here. Both are true, and they are two different records.

INTERCHANGE REFERENCE DATA IS CONTESTED

Cost and interchange reference data is placed here because it varies by instrument class, which satisfies the test. Monetisation has a legitimate claim because it needs the same data to price. Section 18 sets out the argument and the recommendation; it is not settled by the test alone.

Authentication Manager and Fraud

> One product boundary containing two distinct decision points. Keeping them in one boundary is right; letting them become one decision is not.

COMPONENT	SYSTEM OF RECORD FOR	DECISION POINT
3DS / SCA orchestration	Authentication session state, ACS interaction, authentication outcome	D5
Exemption engine	Which exemption was claimed, on what basis, and whether it held	D5
Device and behavioural signals	Device reputation, session behaviour, velocity	Feeds D3
Fraud scoring and models	Model versions, scores and their inputs	D3
Risk rules	Deterministic risk rules and their outcomes	D3
Fraud case management	Investigation state, analyst decisions, confirmed fraud outcomes	Off path

The two decision points differ in kind. **D3 asks how risky this attempt is. D5 asks what to do about it** — proceed frictionless, claim an exemption, or step up. Collapsing them produces a single score-to-action mapping that cannot be tuned for approval and fraud independently, and cannot be experimented on separately.

WHY THIS BOUNDARY IS DRAWN AROUND BOTH

They are one product because the exemption decision needs the fraud score, the fraud model needs authentication outcomes as labels, and splitting them creates a synchronous cross-product call on the authorization path for data that is generated a few milliseconds earlier in the same boundary.

They remain two decision points, two strategy sets and two experiment surfaces. One product, two decisions — and the distinction has to be maintained deliberately, because the natural drift is toward one.

WHAT IS NOT IN THIS BOUNDARY

Merchant credit and underwriting risk is a different discipline with a different time horizon, different data and different regulators. It is a shared service (section 15). Putting portfolio credit risk in the same boundary as per-transaction fraud scoring is a common and expensive confusion.

Dispute and chargeback management is also separate, though it consumes fraud outcomes heavily and feeds labels back.

The eleven named shared services

> Each of these was in the brief. For each: what it uniquely owns, what it must refuse to do, and the boundary most likely to be argued about.

Decision Platform as a Service

Owns. Strategy registration and versioning, flow composition, the binding of one strategy per decision point, deterministic assignment at the divergence point, execution at decision time, and the decision record that every execution emits.

Must refuse. Authoring strategies. The platform executes what product domains author; it does not decide what routing should do, what risk tolerance is acceptable, or which credential is preferred.

Contested edge. Fraud and routing both have mature rules engines today. The migration question is whether those become strategies on this platform or remain separate. If they remain separate there is no shared evidence base and no common definition of a control group, which removes most of the reason to build this.

Experimentation Platform as a Service

Owns. Experiment registration and the mechanical gates, sample size derivation, the allocation ledger, arm assignment, exposure control, treatment fidelity, guardrail evaluation with automatic abort, the permanent holdout, and readouts.

Must refuse. Deciding what to experiment on, or interpreting a result commercially. It produces the number and the interval; the product owner decides what to do about it.

Contested edge. Whether a product may run an experiment outside this platform. The answer has to be no, because the allocation ledger is the only thing preventing two experiments contaminating each other — and a contaminated result is worse than no result.

Global Policy Platform

Owns. The policy set, the authority ladder that resolves conflicts between regulator, network, platform and merchant intent, policy evaluation, and the bitemporal record of what was in force when.

Must refuse. Expressing preference. Policy produces ALLOW, DENY, REQUIRE and BOUND over an option set. “Prefer the cheaper processor” is a strategy, not a policy, and encoding it here removes it from experimentation and makes it invisible to the evidence layer.

Contested edge. The boundary with the Decision Platform. Stated as a rule: policy narrows, strategy chooses within what remains, and a strategy can never widen what policy has closed.

Control Plane and Capability Catalog

Owns. What the platform can actually do, per processor, channel, network, geography and instrument class — verified rather than asserted, with a source and an expiry on each fact. Plus publication: compiling, signing and distributing bundles to every region.

Must refuse. Recording what is sellable or what a client has. Those are two other registries with different lifecycles and different owners.

Contested edge. Everyone wants to write to the capability catalog. It must be writable only by the component that owns the capability, with verification state visible on every entry — otherwise it degrades into a wiki with an API.

Product and Solution Catalog, tied to client entitlement

Owns. The sellable definition: products, solutions, bundles, options, their commercial terms and their dependencies on platform capabilities. Entitlement grants record what a specific client bought, on what terms, effective when.

Must refuse. Enforcing entitlement. Enforcement happens at the edge in L1; the catalog is the source of truth about what was granted, not the gate.

Contested edge. The three-registry separation. Capability is engineering truth, product is commercial truth, entitlement is contractual truth. Collapsing them means no capability can ship until it is priced and sold.

Metering and Monetisation as a Service

Owns. The metering event stream, rating and pricing, rate cards, and the computed amount owed per client per period. Bitemporal, because a rate change and the knowledge of a rate change are different dates.

Must refuse. Moving money or recording a ledger entry. It computes what is owed; the sub-ledger records it and money movement executes it.

Contested edge. Whether metering events are emitted by each product or derived from the event backbone. Emission is explicit and auditable but couples every product to the metering contract; derivation is decoupled but means a billing gap is a data pipeline bug. Recommendation: explicit emission for anything billable, derivation for analytics only.

Settlement Platform

Owns. Funding position, settlement cycles, netting, cut-offs, reserve application, and the settlement instruction. It calculates what is owed to whom and when.

Must refuse. Executing the transfer, and recording the accounting entry. Those are Money Movement and Sub-ledger.

Contested edge. The three-way split with Sub-ledger and Money Movement. Most reconciliation pain in acquiring comes from these three being one system with one team and no internal contract.

Sub-ledger and Reconciliation as a Service

Owns. Double-entry sub-ledger for merchant funds, scheme positions and fee accruals. Reconciliation against scheme clearing files, processor reports and bank statements, with a break management workflow.

Must refuse. Deciding what should have happened. It records and reconciles; settlement decides and money movement executes. A sub-ledger that also calculates settlement can never be used to check settlement.

Contested edge. The interface to the general ledger. The sub-ledger is the system of record for merchant-level positions; the corporate GL receives summarised postings and is not queried for merchant detail.

Currency Management

Owns. FX rate sourcing, rate cards, markup application, quote issue and expiry, DCC, multi-currency pricing and settlement currency selection.

Must refuse. Holding the funds or deciding settlement timing. It prices currency; it does not move it.

Contested edge. Whether DCC sits here or in Payment Processing. DCC is a cardholder-facing offer made at authorization time with strict disclosure rules — the *offer* is a Payment Processing concern at the decision point, the *rate and markup* are Currency Management.

The seven currencies

"Multi-currency support" is not a capability statement. A cross-border transaction passes through up to seven currency contexts, with a conversion, a rate source, a markup and a party bearing the spread at each boundary. Modelling them as one field is how FX margin leaks.

SEVEN CURRENCIES, AND THE CONVERSIONS BETWEEN THEM

Transaction	what the buyer is charged	EUR
Presentment	what the buyer is shown	EUR
FX conversion — rate source, markup, quote, and who bears the spread		
Processing	what the acquirer authorises in	USD
Settlement	what the network settles in	USD
Funding	what lands in our account	USD
Balance	what the client holds with us	USD
FX conversion — rate source, markup, quote, and who bears the spread		
Payout	what the client receives	GBP

— Two conversions in this example, at two different moments, potentially at two different rates, with two different parties bearing the spread. A platform that models only "multi-currency support" cannot answer who paid for either.

FX AS A DECISION POINT

Once the chain above is modelled explicitly, which conversion happens where, at whose rate and with what markup becomes a strategy bound at a decision point rather than a configuration setting. Settlement currency selection, processing currency selection and payout timing all become experimentable.

That is a meaningful product capability and it is not available to a platform that models currency as an attribute of an amount.

Data Platform as a Service

Owns. Ingestion from the event backbone, the canonical analytical model, outcome joins across horizons, feature computation, lineage and retention. The substrate the Decision Platform evidence layer and every readout runs on.

Must refuse. Owning business semantics. A metric definition belongs to the product that owns the concept; the data platform computes and serves it.

Contested edge. Whether the operational decision record is written through the data platform or directly. It must be direct — a decision record that depends on a pipeline is a decision record that can be lost, and this one cannot be reconstructed.

Boarding and Servicing Orchestrator

Owns. The client journey across products: application, underwriting, setup and ongoing servicing changes. The state machine, the sequencing, the dependencies and the client-facing experience.

Must refuse. Owning any product's setup logic. Each product exposes its own setup and servicing steps as contracts; the orchestrator sequences them.

Contested edge. This is the single most likely component to become a monolith. Every product will find it easier to put "just one field" of its own setup logic into the orchestrator. Within two years it contains every product's onboarding rules and no product can change its own boarding without an orchestrator release. Rule 9 exists for this.

The seventeen additions, and why each one is load-bearing

> None of these is speculative. Each is either work already happening without an owner, or a gap that surfaces the first time a real client is boarded in a real jurisdiction.

ADDED COMPONENT	WHY IT IS MISSING-CRITICAL	WHAT HAPPENS WITHOUT IT
Money Movement and Payout	Settlement calculates; something must execute. Payouts, splits, submerchant funding, on-demand pay, reversals and returns.	Each product initiates its own transfers. No single place knows total exposure or can stop payment.
Dispute and Chargeback Management	The full lifecycle: pre-dispute alerts, retrieval, chargeback, representment, pre-arbitration, arbitration, plus deadlines per scheme.	Disputes get handled inside Payment Processing as a state on an attempt, which cannot represent multi-cycle dispute state or scheme deadlines.
Merchant Risk and Underwriting	Credit risk, exposure, reserves, delivery risk, portfolio monitoring, velocity limits at merchant level.	Confused with transaction fraud. Different discipline, different data, different regulator, different time horizon.
Compliance, Screening and Regulatory Reporting	KYC, KYB, UBO, sanctions and PEP screening, AML monitoring, and the reporting obligations that follow.	Boarding cannot complete legally. This is the most common reason a platform cannot enter a new market.
Scheme and Network Management	Mandate intake and tracking, certification, BIN sponsorship, licensing, scheme fee structures, release calendars.	Mandates arrive as surprises and are handled as projects. Every acquirer that has been fined learned this the same way.
Reference and Master Data Management	Merchant hierarchy, legal entity, MID and TID, MCC, country, currency and calendar. The join keys everything else depends on.	Every service invents its own merchant hierarchy and none of them agree at month end.
Cryptographic Services and Key Management	HSM estate, key ceremony and custody, key epochs and rotation, P2PE key injection, the FPE keys the vaultless surrogate derives from.	Key rotation becomes unmanageable. For a vaultless surrogate this is existential — rotating without a key epoch invalidates every token ever issued.
Notification and Communication	Client-facing email, SMS, in-app and statement delivery, with preference and consent management.	Six products each integrate a messaging vendor and clients get six different voices.
Document and Contract Management	Agreements, schedules, amendments, statements, compliance evidence, with retention and e-signature.	The contractual terms behind an entitlement cannot be produced during a dispute or an audit.
Case and Workflow Management	The generic work queue: exceptions, reviews, breaks, escalations, with SLA and audit. Consumed by fraud, disputes, compliance, boarding and recon.	Five products build five case management tools and operations staff use five queues.
Tax Services	Sales tax and VAT determination, 1099-K and equivalent reporting, marketplace facilitator obligations.	A marketplace or PayFac proposition cannot be sold. This is a blocker, not a nice-to-have.
Terminal and Device Management	Estate inventory, remote key injection, firmware and application lifecycle, PCI PTS compliance state, health.	Card present is not a product. You cannot operate a terminal fleet from a spreadsheet.
Entitlement Service	The grant store behind the enforcement point — what this client bought, on what terms, effective when, with what limits.	Entitlement is inferred from configuration scattered across products and drifts from the contract.
Tenancy and Residency Router	Resolves legal entity, region and residency obligations once, at admission.	Every component interprets residency independently and the answer differs by component.
Data Residency and Sovereignty Controls	Enforcement and proof that data stayed where it was required to stay.	Market entry blocked, or entered on an assertion that cannot be evidenced.
Resilience and Continuity	Regional failover, degradation ladder, stand-in processing, stated recovery objectives per component.	Availability is an emergent property nobody owns, discovered during the first regional incident.
Sandbox and Simulator Fabric	Deterministic processor, network and issuer simulators and reproducible test data. Infrastructure rather than business capability, but the same argument.	Every integration test depends on a third party's test environment behaving consistently, which is the most reliable way to make a release train unpredictable.

THE PATTERN IN THE SEVENTEEN

Twelve of the seventeen are either compliance, money movement or master data. That is not a coincidence — those are the capabilities that are invisible while the platform is a prototype and non-negotiable the moment it is a regulated business with real clients in real jurisdictions.

The practical consequence is that these cannot all be deferred to a later phase. Compliance and screening gate the first boarding; money movement gates the first settlement; master data gates the first month-end. They are early, not late.

Master component register

> 71 internal components, plus six classes of external counterparty. 17 carry a `+`, meaning they were added beyond the original brief. The “system of record for” column is the important one — it is the operative form of principle 1, and a component with nothing in that column should be questioned.

COMPONENT	L	BOUNDARY	SYSTEM OF RECORD FOR	+
Merchant Systems ecom · OMS · ERP	L0	External	— outside the boundary	
Checkout & Payment Element	L0	Payment Processing	Collection session state	
Mobile & In-App SDK	L0	Payment Processing	SDK capability matrix and version	
Terminal & POS Estate	L0	Payment Processing	Card-present transaction session	
Client Portal & Reporting	L0	Shared	Nothing — presentation only	
Developer Portal & Sandbox	L0	Shared	Published API surface and sandbox keys	
API Gateway version · limits	L1	Shared	Admission policy, rate limits, API versions	
Client Identity & Access	L1	Shared	Client credentials, scopes, roles	
Entitlement Enforcement	L1	Shared	Nothing — enforcement point only	
Idempotency Request Ledger	L1	Payment Processing	That a request was received and what it resolved to	
Webhook & Event Delivery	L1	Shared	Delivery state, ordering, retry and replay position	
Tenancy & Residency Router	L1	Shared	Legal entity, region and residency resolution	+
Order · Payment · Session · Attempt	L2	Payment Processing	The canonical transaction resource model and every state transition	
Orchestration (UPOP)	L2	Payment Processing	Flow execution and transaction context	
Routing & Retry	L2	Payment Processing	Routing and retry strategies (executed by the Decision Platform)	
Capture · Refund · Void	L2	Payment Processing	Post-authorization money operations against an attempt	
Network & Processor Connectivity	L2	Payment Processing	Per-connector capability declaration and session state	
APM & Local Rails	L2	Payment Processing	Non-card method orchestration and rail semantics	
Surrogate Service (vaultless)	L2	Credential Vault	Nothing — a pure keyed function with a key epoch	
Credential State Service	L2	Credential Vault	Token status, expiry, provisioning state, channel provenance	
Network Tokenisation (TSP)	L2	Credential Vault	TSP relationship and token lifecycle events	
Account Updater	L2	Credential Vault	Latest known good credential and update history	
Consent & Mandate	L2	Credential Vault	What the cardholder agreed to, when, in which channel	
Identity Resolution (PAR)	L2	Credential Vault	Mapping from captured credential to stable identity	
BIN & Range Intelligence	L2	Instrument Intelligence	BIN ranges, product codes, issuer mapping over time	
Issuer Intelligence	L2	Instrument Intelligence	Observed issuer behaviour and authentication posture	
Instrument Classification	L2	Instrument Intelligence	Credit, debit, prepaid, commercial, consumer, regulated status	
Reachability & Eligibility	L2	Instrument Intelligence	Which networks, rails and token services an instrument reaches	
Regulatory Attributes	L2	Instrument Intelligence	Regulated debit status, issuer country, scheme classifications	
Cost & Interchange Reference	L2	Instrument Intelligence	Interchange qualification reference and scheme fee structures	
3DS / SCA Orchestration	L2	Authentication & Fraud	Authentication session state and outcome	
Exemption Engine	L2	Authentication & Fraud	Exemption claimed, basis, and whether it held	
Device & Behavioural Signals	L2	Authentication & Fraud	Device reputation, session behaviour, velocity	
Fraud Scoring & Models	L2	Authentication & Fraud	Model versions, scores and their inputs	
Risk Rules	L2	Authentication & Fraud	Deterministic risk rules and outcomes	
Fraud Case Management	L2	Authentication & Fraud	Investigation state and confirmed fraud outcomes	
Decision Platform as a Service	L3	Shared	Strategies, flows, bindings, assignment and the decision record	

Experimentation Platform as a Service	L3	Shared	Experiments, the allocation ledger, exposure, holdout and readouts	
Global Policy Platform	L3	Shared	The policy set, authority ladder and what was in force when	
Control Plane & Capability Catalog	L3	Shared	Verified platform capability, and bundle publication	
Product & Solution Catalog	L3	Shared	Sellable definitions, terms and capability dependencies	
Scheme & Network Management	L3	Shared	Mandates, certification state, licensing, scheme fee structures	+
Settlement Platform	L3	Shared	Funding position, cycles, netting, cut-offs, settlement instruction	
Sub-ledger & Reconciliation	L3	Shared	Double-entry positions and reconciliation break state	
Money Movement & Payout	L3	Shared	Transfer instruction, execution state, exposure	+
Currency Management (FX)	L3	Shared	FX rates, markup, quotes and their expiry	
Metering & Monetisation	L3	Shared	Metering events, rate cards, rated amounts (bitemporal)	
Tax Services	L3	Shared	Tax determination results and reporting obligations	+
Boarding & Servicing Orchestrator	L3	Shared	Client journey state across products	
Merchant Risk & Underwriting	L3	Shared	Credit exposure, reserves, limits, portfolio monitoring	+
Compliance, Screening & RegRep	L3	Shared	KYC/KYB/UBO outcomes, screening results, regulatory filings	+
Dispute & Chargeback Management	L3	Shared	Dispute lifecycle state and scheme deadlines	+
Case & Workflow Management	L3	Shared	Work items, assignment, SLA and audit trail	+
Document & Contract Management	L3	Shared	Agreements, schedules, statements, retention	+
Data Platform as a Service	L3	Shared	Analytical model, outcome joins, features, lineage	
Reference & Master Data Management	L3	Shared	Merchant hierarchy, legal entity, MID/TID, MCC, country, currency	+
Cryptographic Services & KMS	L3	Shared	Key material, epochs, ceremony and custody records	+
Notification & Communication	L3	Shared	Delivery state, preferences and consent	+
Terminal & Device Management	L3	Shared	Estate inventory, firmware, key injection, PTS compliance state	+
Entitlement Service	L3	Shared	Grants: what a client bought, on what terms, effective when	+
Connector & Adapter Fabric	L4	Payment Processing	Wire translation, certification state, connector versions	
Network Iface ISO 8583 · 20022	L4	Payment Processing	Scheme session state and message construction	
Event Backbone	L4	Shared	Ordered durable event log	
File & Batch Transfer	L4	Shared	Transfer state and file receipt	
Partner Gateway	L4	Shared	Partner integration state	
Sandbox & Simulator Fabric	L4	Shared	Simulator behaviour and reproducible test data	+
Observability & SRE	L5	Shared	Telemetry, traces, SLO state	
Platform Runtime & Service Mesh	L5	Shared	Service identity and traffic topology	
Secrets & Config	L5	Shared	Secret material and rotation state	
CI/CD & Change Mgmt	L5	Shared	Code change history and approvals	
Data Residency & Sovereignty	L5	Shared	Residency obligations and proof of compliance	+
Resilience & Continuity	L5	Shared	Recovery objectives and degradation state	+

Twelve boundary rules

> A boundary that is drawn but not defended is gone within two quarters. These are the rules that do the defending, written so that a violation is observable rather than a matter of opinion.

#	RULE	HOW A VIOLATION IS DETECTED
1	No component reads another boundary's datastore. Cross-boundary access is a versioned contract.	Network policy and datastore credentials scoped per boundary. A violation fails to connect.
2	Each fact has exactly one system of record. Everyone else holds a cache with a declared staleness tolerance.	The register in section 16 is the list. A second component claiming the same fact is an architecture review item.
3	No product domain ships its own decision engine. Strategies are registered with the Decision Platform.	Any rules evaluation on the request path outside the Decision Platform appears as an untracked decision in the evidence completeness check.
4	Policy narrows, strategy chooses. A strategy may never widen a policy-restricted option set.	The decision record shows the permitted set before and after strategy evaluation. Widening is detectable per decision.
5	No product domain initiates money movement, writes a ledger entry, or issues a payout.	Money Movement is the only component with bank and payout credentials.
6	Capability, product and entitlement are three separate registries with three separate write paths.	A capability entry that cannot be created without a price is the symptom.
7	Entitlement is enforced once, at the edge. Products do not re-check entitlement.	An entitlement check inside a product domain is a code review reject.
8	Every decision on the request path emits a decision record with propensity and fallback.	Daily reconciliation of decision records against eligible transactions, at or above 99.5%.
9	The Boarding Orchestrator sequences; it does not contain product setup logic.	Any product-specific conditional inside the orchestrator. Measured as orchestrator releases caused by a single product's change.
10	No component outside the payment element, the terminal estate and the connector fabric accepts a PAN.	Automated scanning of logs, payloads and fixtures. This one decays silently and needs continuous enforcement.
11	Anything contested is bitemporal: pricing, policy, entitlement, capability, rates.	The schema either has valid time and knowledge time, or it does not.
12	Forking a shared service requires a named architectural exception with an expiry date.	A register of exceptions. An empty register over a long period usually means the rule is being ignored rather than observed.

RULE 12 IS THE ONE THAT MATTERS MOST

Eleven of these rules constrain product teams. The twelfth constrains the platform organisation, and it is the one that makes the other eleven legitimate.

A product team that forks a shared service is almost always responding to a real problem: the shared service could not move at the pace the roadmap needed. Making the fork visible and time-boxed turns that into a capacity conversation. Forbidding it outright, with no exception path, guarantees the fork happens anyway and is simply not called one.

The nine contentious boundaries

> These are the seams where reasonable, well-informed people will disagree. Each one is stated with the argument on both sides and a recommendation, because leaving them implicit means they get settled by whoever ships first.

1. Does Fraud keep its own decision engine?

One view. Fraud decisioning is mature, latency-critical, and its vendor and model tooling assume a self-contained engine. Migrating it to a general decision platform risks a working system for architectural tidiness.

The other. If fraud stays separate there are two definitions of a control group, two assignment mechanisms and two evidence stores. Nothing can be measured across the boundary, which removes most of the reason to build a decision platform at all.

Recommendation. Fraud ***models*** stay with Fraud. Fraud ***strategies*** — the mapping from score to action at D3 — register with the Decision Platform. That keeps the model estate intact while making the decision experimentable. Accept that this is the hardest migration in the programme.

2. Where does interchange reference data live?

One view. It varies by instrument class, which puts it in Instrument Intelligence by the section 5 test, and routing needs it at decision time.

The other. Monetisation needs exactly the same data to price and to compute what is owed, and a second copy will drift from the first.

Recommendation. Instrument Intelligence is the system of record; Monetisation consumes it as a contract rather than maintaining a copy. Interchange is an attribute of the instrument and the transaction — pricing is a function applied to it.

3. Settlement, Sub-ledger and Money Movement — one team or three?

One view. They are operationally inseparable, share a domain vocabulary, and splitting them adds interfaces in the most latency-tolerant part of the platform for no user benefit.

The other. A sub-ledger that also calculates settlement cannot be used to check settlement. The separation is what makes reconciliation meaningful rather than circular.

Recommendation. Three components with hard contracts, which may be one team. The contract boundary is what matters, not the org chart. If they are one team, the internal interface still has to be real enough that the ledger can contradict settlement.

4. Is the exemption decision Authentication or Payment Processing?

One view. It is made at D5 inside the authentication flow using the fraud score, so it belongs with Authentication.

The other. The commercial consequence — liability shift, approval impact, cost — is a payment outcome, and the merchant experiences it as a payment behaviour.

Recommendation. Authentication owns the exemption decision. Payment Processing owns the objective it is decided against. This is the same shape as routing: the product owns the goal, the specialist owns the mechanism.

5. Does the Credential Vault own PAR, or does Instrument Intelligence?

One view. PAR is issued by the network per underlying account, which makes it look like class-level reference data.

The other. PAR identifies a specific cardholder account, not a class. It varies per cardholder, so the section 5 test puts it in the Vault.

Recommendation. Credential Vault. PAR is the cross-channel identity anchor and belongs with identity resolution. Instrument Intelligence knows whether a BIN ***supports*** PAR; the Vault knows what it ***is*** for a given credential.

6. Who owns the merchant hierarchy?

One view. Boarding creates it, so Boarding should own it. Every other consumer is downstream of onboarding.

The other. Boarding is a workflow, not a system of record. Hierarchy changes long after boarding — acquisitions, restructures, new outlets — and every one of settlement, risk, pricing and reporting depends on it.

Recommendation. Reference and Master Data Management owns it. Boarding is the first writer, not the owner. This is the single most common master data failure in acquiring, and it surfaces at month end.

7. Should metering events be emitted or derived?

One view. Deriving them from the event backbone decouples every product from the metering contract and means no product can forget to emit.

The other. Derivation makes a billing gap a data pipeline bug, and pipeline bugs are found late. Explicit emission is auditable and testable per product.

Recommendation. Explicit emission for anything billable; derivation for analytics only. Revenue accuracy beats coupling avoidance, and a missed metering event is a revenue leak nobody detects.

8. Is Dispute Management part of Payment Processing?

One view. A dispute is a state of a payment, and modelling it elsewhere creates a distributed state machine over the same transaction.

The other. Dispute lifecycle is multi-cycle with scheme-specific deadlines, evidence handling and representment workflow. Expressing that as a state on an attempt

forces every dispute concept into the payment model.

Recommendation. Separate shared service holding dispute lifecycle state, linked to the payment by reference. Payment Processing holds the financial consequence of a dispute; it does not hold the case.

9. Does the Control Plane own publication, or does each service?

One view. Each service knows best how to distribute its own configuration and has different freshness needs.

The other. Multiple publication paths mean multiple consistency models, and a transaction can then be evaluated against a policy from one epoch and a strategy from another.

Recommendation. One publication path, one signed bundle, one epoch. Per-service publication is how you get a flow resolved against a capability set that no longer matches the policy set — which is unreproducible and unexplainable after the fact.

Runtime — one authorization, edge to issuer

> The architecture as a transaction experiences it. Nine steps, four boundaries, and eight decision points — all of it under a latency budget that does not care how many components are involved.

ONE CARD-NOT-PRESENT AUTHORIZATION, EDGE TO ISSUER

1 Request
API gateway, idempotency key, tenancy and residency route
SH

2 Entitle
Checked against what this client actually bought
SH

3 Resolve
Credential Vault returns PAN surrogate or network token
CV

4 Enrich
Instrument Intelligence: BIN, issuer, eligibility, cost
PII

5 Constrain
Global Policy Platform narrows the permitted option set
SH

6 Decide
Decision Platform runs the strategy bound at each point
SH

7 Authenticate
Auth Manager and Fraud: score, exempt or step up
AF

8 Dispatch
Connector fabric to processor, network and issuer
PP

9 Record
Decision record, metering event, sub-ledger entry
SH

DECISION POINTS ON THIS PATH

D1 Validate	**D2** Enrich	**D3** Risk	**D4** Gate
D4.1 Credential	**D4.2** Retry	**D5** Authenticate	**D6** Route

— The product domain owns the strategy. The Decision Platform owns the execution, the assignment and the evidence.
— Every decision emits what was chosen, what else was permitted, the propensity, and what the fallback would have done.
— Policy narrows the option set before any strategy runs. A strategy can never widen what policy has closed.
— No step reads another product's database. Every hop is a versioned contract.

What the path implies about availability

Every synchronous hop on this path is an availability multiplier. Five services at 99.95% each is 99.75% composite — roughly nine times the downtime of any single one. That arithmetic is why the architecture pushes as much as possible off the synchronous path:

MECHANISM	WHAT IT REMOVES FROM THE PATH	RESIDUAL RISK
Bundle publication	Policy, strategy, flow and capability lookups become local resolution	Bundle staleness. Bounded and measured; degrades to last-known-good.
Vaultless surrogate	The majority of credential resolutions become a keyed derivation, not a lookup	HSM availability, which is a different and better-understood failure domain.
Cached instrument intelligence	BIN and issuer attributes served from a local cache with a stated tolerance	A newly issued BIN range is unknown for the tolerance window.
Asynchronous evidence	Decision records, metering events and ledger entries are emitted, not called	Emission loss. Reconciled daily; a gap invalidates readouts for that window.
Degradation ladder	Each dependency has a defined behaviour when unavailable	The ladder has to be rehearsed. An untested degradation path is an outage with extra steps.

The latency budget

The question a top-tier PSP is judged on is not how many services it runs. It is how much latency the platform itself adds before the network receives the authorization. That number has to be a budget with named owners, not an emergent property.

STAGE	p99 BUDGET	OWNER	NOTE
Edge admission — gateway, identity, entitlement, tenancy	8 ms	Shared	Bundle-resolved; no remote calls
Credential resolution	12 ms	Credential Vault	The one hard synchronous dependency
Instrument enrichment	3 ms	Instrument Intelligence	Local cache with stated staleness
Policy evaluation	2 ms	Shared	Local, from the published bundle
Risk scoring	18 ms	Auth & Fraud	The largest single line, and the most at risk
Authentication decision	6 ms	Auth & Fraud	Excludes external 3DS entirely
Routing decision	3 ms	Payment Processing	Local strategy evaluation
Decision platform overhead	4 ms	Shared	Assignment, binding resolution, record emission
Connector serialisation and internal transit	12 ms	Payment Processing	To the point the message leaves us
Platform-added total	**68 ms**	—	**Against a stated ceiling of 100 ms**

HOW TO READ THIS BUDGET

The total is the ***sum of per-stage p99***, which is deliberately conservative — the composite p99 of independent stages is lower than the sum of their individual p99s. Budgeting on the sum leaves headroom rather than consuming it.

Two lines will breach first. **Risk scoring at 18 ms** is optimistic if any vendor call sits inside it; a synchronous third-party score does not fit this budget and has to be either pre-computed or made asynchronous with a fallback. **Credential resolution at 12 ms** is the hard synchronous dependency and the only line that cannot be engineered away.

External 3DS is excluded on purpose. A challenge flow adds hundreds to thousands of milliseconds and is a separate budget with a separate conversation about when it is worth it.

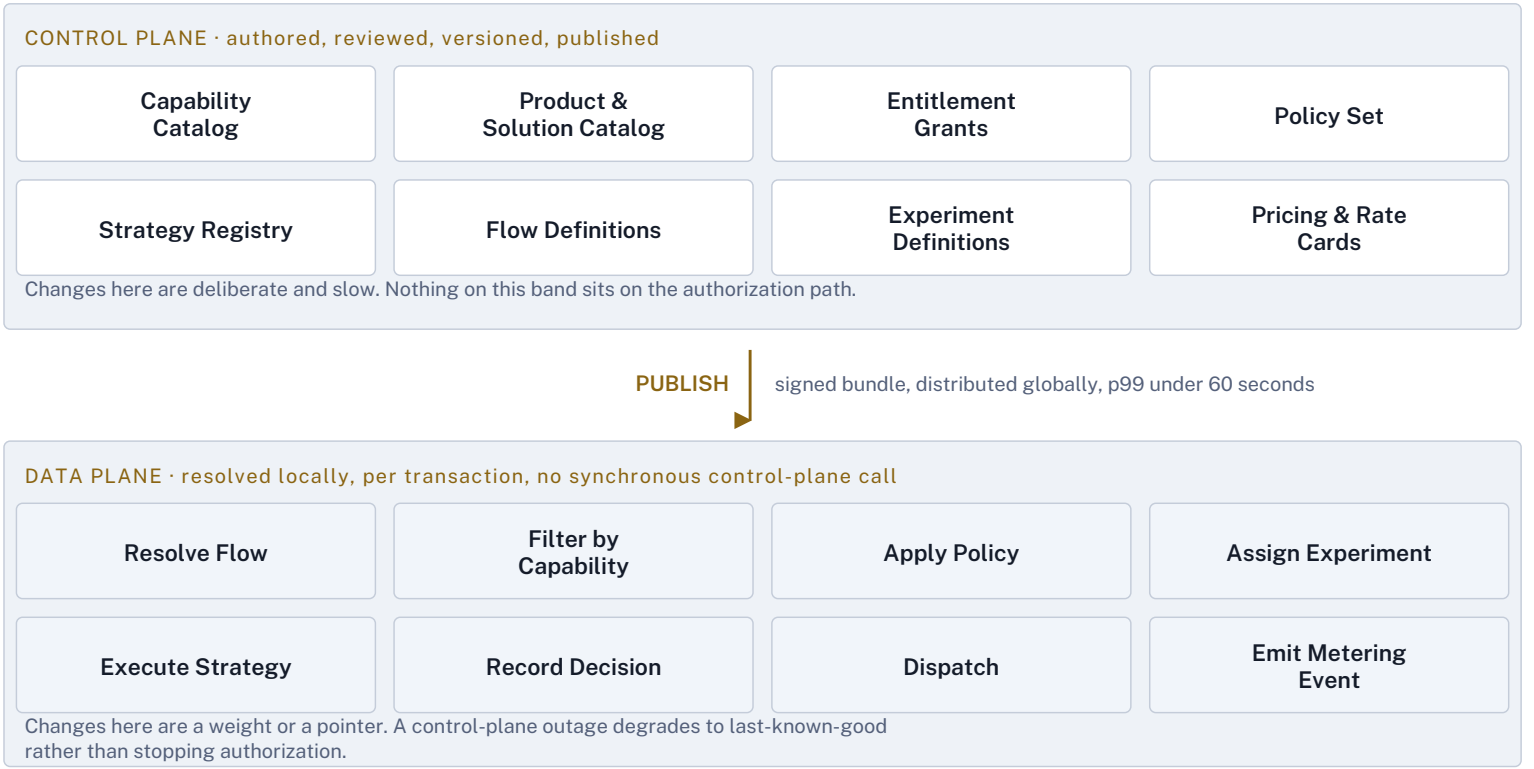
THE ONE HOP THAT CANNOT BE REMOVED

The credential state lookup for a network token cannot be derived, cached indefinitely, or made asynchronous — token status is assigned by the network and changes without warning. It is the hard synchronous dependency in this architecture and it should be engineered as such: its own availability target, its own regional replication, and a defined behaviour when it is unreachable.

That behaviour is a product decision, not an engineering one. Falling back to the PAN when token status is unknown trades a small fraud and interchange cost for availability. Failing the transaction trades availability for correctness. Someone has to own that trade and write it down.

Control plane and data plane

> The separation that everything else rests on. It is what lets a routing change take days instead of a quarter, and what stops a configuration outage becoming an authorization outage.



— The separation is the architecture. Everything slow, reviewed and contested lives above the line. Everything fast and reversible lives below it.

	CONTROL PLANE	DATA PLANE
Changes	Authored, reviewed, versioned	A weight or a pointer
Pace	Deliberate and slow	Immediate
Governed by	Whoever owns the business decision	Exposure authority
Failure mode	Publication stops; running traffic is unaffected	Authorization stops
Availability target	High, but not authorization-grade	Authorization-grade
Consistency	Eventually consistent, epoch-versioned	Locally consistent within an epoch
On the request path	Never	Always

What the epoch binds together

BOUND INTO THE EPOCH	WHY IT MUST BE IN THE SAME VERSION
Policy set	A decision evaluated against a newer policy than the strategy assumed is not reproducible
Capability set	A strategy may select an option the capability set no longer permits
Strategy registry	The bound strategy version must be the one that was reviewed
Flow definitions	The binding at each decision point is part of the flow, not the strategy
Pricing and rate cards	A transaction rated under a different epoch than it was decided under is a dispute
Entitlement grants	What the client was entitled to at the moment of the decision, not now

With all six in one signed epoch, a decision record can answer the question that matters in a dispute, an audit, a merchant escalation or a model evaluation: **exactly why did we make this decision, and what was true when we made it.** That answer is worth more than any single optimisation the platform will produce.

THE EPOCH IS WHAT MAKES THIS COHERENT

A transaction must be evaluated against one consistent set of policy, capability, strategy and flow — not against whichever version of each happened to be resolvable at that instant. One signed bundle carrying an epoch identifier gives that property, and the epoch on the decision record is what makes a decision reproducible months later.

Without it you get the failure that is almost impossible to debug: a transaction evaluated against a policy from Tuesday and a capability set from Thursday, producing an outcome that cannot be reproduced and cannot be explained.

Six anti-patterns this design exists to prevent

> Each of these is a thing that happens by default. The architecture is, in large part, the set of decisions taken specifically to make them harder.

ANTI-PATTERN	HOW IT ARRIVES	WHAT PREVENTS IT HERE
The orchestrator monolith. One service ends up containing every product's business logic because it is the only thing that sees the whole flow.	Each product adds "just one field" of its own setup or routing logic to the orchestrator, because that is where the sequencing already is.	Rule 9. The orchestrator sequences contracts; products own their own steps. Measured by orchestrator releases caused by a single product's change.
The shadow BIN table. The orchestration layer starts caching instrument data, then owning it, then diverging from the source.	A latency problem is solved with a local cache, which then needs a refresh policy, which then needs an owner.	Rule 2 plus a declared staleness tolerance per cached fact, with distribution owned by Instrument Intelligence rather than by the consumer.
Four decision engines. Routing, fraud, authentication and credential selection each build their own rules engine.	Each is individually reasonable and locally faster than adopting a shared platform.	Rule 3. The cost is not four engines — it is four definitions of a control group and no shared evidence, which makes cross-domain optimisation impossible.
Entitlement drift. What a client can do diverges from what they bought.	Entitlement is enforced by configuration inside each product, and configuration is edited by operations to fix an incident.	Rules 6 and 7. One grant store, one enforcement point at the edge, and no entitlement logic inside a product domain.
Policy as preference. Regulatory constraints get encoded as routing strategies because that is where the mechanism already exists.	"Do not route this BIN to that processor" is expressible as a strategy, and shipping it that way is faster than modelling it as policy.	Rule 4 and a policy vocabulary that cannot express preference. If it is not ALLOW, DENY, REQUIRE or BOUND, it is not policy.
Reconciliation by inference. Nobody records what was decided, so month-end breaks are reconstructed from logs and argument.	Metering and ledger entries are derived from pipelines rather than emitted explicitly, and the pipeline is correct until it is not.	Rule 5 and explicit emission of billable events. Settlement calculates, the sub-ledger records, money movement executes, and the three can contradict each other — which is the point.

Open decisions

> Nineteen decisions. Nine are architectural and are resolved here; ten are strategic and are resolved in Volume 2. All of them need a named owner and a date, because each blocks something concrete and none resolves itself.

Nine architectural decisions

#	DECISION	BLOCKS	OWNER
1	Does fraud decisioning migrate to the Decision Platform, and on what timeline?	Any cross-domain optimisation; any joint credential-and-risk experiment	Product · Architecture
2	Is the platform organisation funded to own L1, L3, L4 and L5 as a product?	Everything. Two-thirds of this architecture has no owner otherwise	Technology leadership
3	What is the degradation behaviour when credential state is unreachable — fall back to PAN, or fail?	The availability target for the whole authorization path	Product · Risk
4	Is the vaultless surrogate tweak scoped per merchant or per platform?	Cross-merchant omnichannel identity. A one-way door — changing it re-tokenises the estate	Product · Security
5	Which capability facts must be processor-verified and which may be asserted with an expiry?	Capability registry delivery; the honesty of the capability filter	Architecture
6	Does the sub-ledger post to the corporate GL in summary or in detail?	Finance integration and the merchant-detail query path	Finance · Platform
7	Which markets are in scope for residency enforcement in the first release?	Tenancy router and residency control scope	Product · Compliance
8	Who owns the merchant hierarchy on day one — Reference Data or Boarding?	Settlement, pricing, risk and reporting all join on it	Architecture · Operations
9	Is there an exception register for rule 12, and who approves entries?	Whether the shared services survive contact with four roadmaps	Architecture

Ten strategic decisions, deferred to Volume 2

These are product and commercial decisions rather than architecture decisions. They are recorded here because the architecture constrains them and they constrain the architecture, but they are resolved in Volume 2 rather than in this document.

#	DECISION	WHY IT CANNOT BE DEFERRED INDEFINITELY
10	Who owns Payment Intent versus Order semantics?	Every API surface depends on it. Changing it later is a breaking change for every integration.
11	What is the canonical payment method abstraction?	Determines whether adding a rail is a quarter or a year.
12	What is the canonical multi-currency model?	FX margin leaks quietly until the model is explicit.
13	Where does recurring billing live — a product domain or a composition?	Decides whether revenue recovery is a product or a feature.
14	Where does marketplace and split-payment orchestration live?	Blocks the platform and PayFac proposition entirely.
15	What is the global acquiring model — own licences, sponsored, or hybrid by market?	Determines settlement, compliance and entity architecture per market.
16	What is the SLO for platform-added authorization latency?	Without a number the budget above is advisory.
17	What is the Decision Platform feature and model architecture?	Determines whether learned strategies are possible at all.
18	How does counterfactual evaluation work, and who validates it?	Decides whether a model can reach production safely.
19	What is the agentic commerce trust model?	The primitives are schema decisions. Retrofitting them is the propensity-logging problem again.

THE TWO THAT ARE ACTUALLY URGENT

Decision 2 is urgent because it is structural. Forty-two of the components here are shared, and a product-shaped organisation leaves them orphaned or carved up. Every other decision on this page assumes someone owns the platform as a product.

Decision 4 is urgent because it is a one-way door. The tweak scope for the vaultless surrogate determines whether cross-merchant omnichannel identity is possible at all, and changing it later means re-tokenising every stored credential in the estate. It should be decided deliberately, now, rather than defaulted into by whoever implements first.